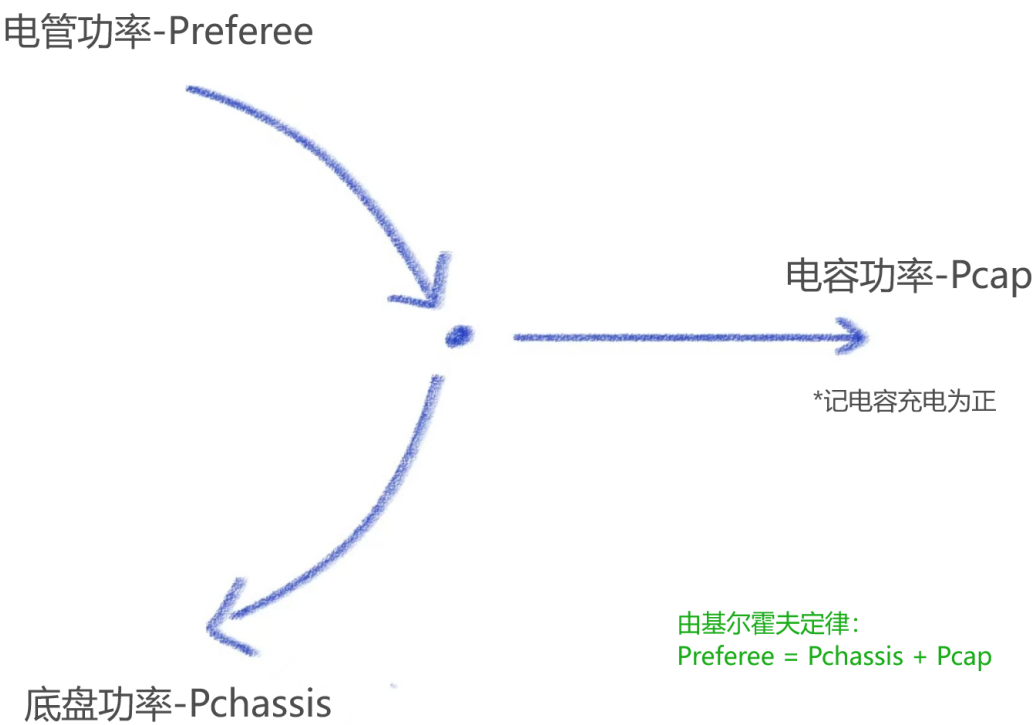


超电CAN通信 V0.5

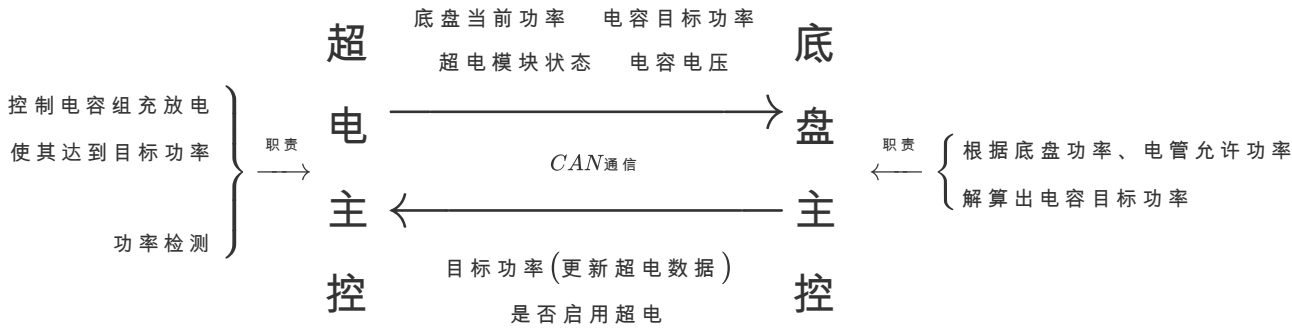
一 基本逻辑

1.1 电路原理



增加了电容的底盘电路可以理解成上面的**三端网络**，由电管、底盘、电容三个端口组成，由基尔霍夫定律可知，电管功率等于底盘功率与电容功率之和；这个关系对于电容放电也是成立的，此时图中电容功率为负，底盘功率比电管功率大。

1.2 控制实现



超电发送的数据有底盘当前功率和电容目标功率。底盘当前功率是底盘电机消耗的功率，作为底盘主控那边的功率检测输入使用（新赛季要求裁判系统不给底盘功率数据，必须超电模块检测），电容目标功率就是底盘主控发送过来更新的功率，指的是电容充/放电的功率，用于检验超电这边确实更新了。超电模块状态目前只是个雏形，后面完善。电容电压指的是电容组当前剩余的电压，也就是指示电容组剩余能量。

关于当前功率-缓冲能量-限制功率之间关系的计算，有学校在超电主控上完成也有在底盘主控上完成，因为你们底盘已经有功率控制代码了，暂且设定为底盘那边计算，给超电只需要发送一个超电需要吸收/供给的目标功率即可。这里的基本逻辑是用电管允许的功率减去底盘实际消耗的能量（由上面的三端网络图可知），得到剩余的功率就是允许超电此时趁机吸收的能量；反之如果相减的值为负值，也就是说底盘消耗的功率比电管允许功率大，那么超电往外辅助放电，保证电管功率不超过限额。

二 数据格式

2.1 CAN通信配置

波特率: 1MBit/s

超电使用CAN ID: 0x211

底盘主控请使用CAN ID: 0x210

发送频率: 其使用的定时器中断频率是193Hz,实际发送没有测过,这个方便调节的.

2.2 超电发送数据格式

发送的数据需要使用四个元素的uint16_t数组接收；但CAN通信是以一个字节（即8位）为单位发送数据的，因此一般来说CAN发送函数会要求你的发送数据数组只能放uint8_t类型。那么有下面两种解决方案。（注意旧版本文档对于高位在前和低位在前的逻辑搞错了，请以文档最新版为准）

底盘主控接收代码示例（使用强制类型转换）

对uint16_t数组直接进行**强制类型转换**,四个元素的uint16_t转换过来就是八个元素的uint8_t，然后按照正常的长度为8的CAN数据发送即可。

```
1 //接收数组定义
2 uint16_t bsp_can_rxbuf[4];
3
4 //下面是接收响应函数中的代码
5 HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO1, &rxconf, (uint8_t *)bsp_can_rxbuf);
6 //这是我的程序，底盘使用的时候hcan, FIFO, header等按照自己的写！
```

```
64 uint16_t bsp_can_rxbuf[4]; ← 接收数组
65 uint16_t rxl = 0;
66 void HAL_CAN_RxFifo1MsgPendingCallback(CAN_HandleTypeDef *hcan) {
67     CAN_RxHeaderTypeDef rxconf;
68     HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO1, &rxconf, (uint8_t *)bsp_can_rxbuf);
69     rxl++;
70     HAL_GPIO_TogglePin(GPIOB, LED_G_Pin); //绿灯 强制类型转换
```

底盘主控接收代码示例（使用低位在前逻辑）

这里的强制类型转换在效果上与低位在前的数据存储方式是一样的，C620电机是高位在前，与此相反；电控直接把读取C620的代码复制过来，把两个uint8_t变量的顺序倒换过来，变成低位在前，是一样的效果，全向用的就是这个方式读取，总之两种接收方式，二选一即可，效果都一样。

```
1 //存放数据的结构体的定义
2 typedef struct
3 {
4     uint16_t Chassis_Power; //底盘当前功率
5     uint16_t Set_Power;     //电容目标功率
6     uint16_t Referee_Power; //电管当前功率
7     uint16_t Cap_Voltage;   //电容当前电压
8 }PowerCtrl;
9 PowerCtrl PowerData;
10
11 //下面是接收响应函数中的代码
12 if(RX_message.StdId==0x211) //若CAN信息来自超电
13 {
14     //采用低位在前
15     PowerData.Chassis_Power = (RX_message.Data[1] << 8|RX_message.Data[0]); //底盘当前功率
16
17     PowerData.Set_power = (RX_message.Data[3] << 8|RX_message.Data[2]); //电容目标功率
18
19     PowerData.Referee_Power = (RX_message.Data[5] << 8|RX_message.Data[4]); //电管当前功率
20
21     PowerData.Cap_Voltage = (RX_message.Data[7] << 8|RX_message.Data[6]); //电容当前电压
22 }
```

建议按照示例代码，这样方便统一。

接收效果大概像这样（数值仅供示意，重点是这里是四个uint16_t类型数据）：

Name	Value	Type
stop	<cannot evaluate>	uchar
bsp_can_rxbuf	0x2000006A bsp_can_r...	ushort[4]
[0]	0	ushort
[1]	32762	ushort
[2]	24	ushort
[3]	65535	ushort
<Enter expression>		

内容

CAN报文的四个元素按顺序分别为：

- 1. 底盘当前功率
- 2. 目标设定功率
- 3. 电管当前功率
- 4. 电容电压

下面是数据的详细解析。

底盘当前功率

即上面电路原理中的Pchassis.

因为使用的都是uint16_t类型的元素，所以不能直接表示浮点数和负数。超电主控把数据如下处理：

$$y = x * 10 + 0x7FFF$$

其中x,y分别是原数据和发送数据，为了表示一位小数把x乘10，为了表示负数把它的零点”居中“在十六进制0x0000到0xFFFF的一半，即0x7FFF。

底盘主控接收到数据y之后，应对其反变换：

$$x = (y - 0x7FFF)/10$$

即可获得原始发送数据x，即带正负，保留一位小数的底盘功率（注意及时转换成float以免丢失）

电容目标功率

即上面电路原理中Pcap的目标值。

同底盘功率进行相同的从x到y的变换。

电管当前功率

即上面电路原理中的Preferee.

同底盘功率进行相同的从x到y的变换。

超电状态（V0.5改动：该数据暂不发送

这里用状态码的思路进行表示超电状态。

数值	状态	具体意思
0xFFFF	未初始化	此为超电状态码初始值，没有变成别的说明还没初始化跑起来
0x0001	正常充电	超电状态正常且电容组正在充电(功率>0)
0x0002	正常放电	超电状态正常且电容组正在放电(功率<0)
0x0003	正常待机	超电状态正常且电容组不参与底盘工作(不上电容状态)
0x0005	过压过流过充过放保护	电压或电流过大，或者超电充满了或者耗尽了，自动停机
0x0009	高温保护	超电上有测温用的NTC，后面做高温保护

可能还有别的状态，我还没想好（

总之我设计的可能花样多一点，底盘那边看需要就用。

电容电压

即电容组电压，反映超级电容组剩余电量

使用的变换为

$$y = x * 100$$

类似地，乘以100是为了保留两位小数。

底盘主控在接收到数据 y 之后，应对其作反变换：

$$x = y/100$$

以获得原始数据 x 。（注意及时转换为float类型）

对数据进行反变换的代码示例

在使用上面那个示例代码进行数据接收后,需要对四个(实际上是三个)数据进行反变换,得到浮点数.

```
1 //实际数据结构体定义
2 typedef struct
3 {
4     float Chassis_Power;    //底盘当前功率
5     float Set_Power;        //电容目标功率
6     float Referee_Power;    //电管当前功率
7     float Cap_Voltage;      //电容当前电压
8 }PowerCtrl_True;
9 PowerCtrl_True PowerData_True;
10
11 //反变换函数
12 void Cap_Data_Decode(void)
13 {
14     PowerData_True.Chassis_Power = (PowerData.Chassis_Power - 0x7FFF) / 10.0f; //底盘当前功率
15
16     PowerData_True.Set_Power = (PowerData.Set_Power - 0x7FFF) / 10.0f; //电容目标功率
17
18     PowerData_True.Referee_Power = (PowerData.Referee_Power - 0x7FFF) / 10.0f; //电管当前功率
19
20     PowerData_True.Cap_Voltage = PowerData.Cap_Voltage / 100.f; //电容当前电压
21 }
```

2.3 超电接收数据格式

内容

底盘发送数据按顺序依次为:

1. 电容目标功率
2. 未定

3. 未定
4. 未定

电容目标功率

即给超电模块发送超电要充电的目标功率（负值即为放电），发送过来后超电由此值进行更新自己的电容目标功率，为了避免超电模块PID的超调/过冲导致进入保护，甚至烧坏板子（亲身经历），请不要使得您发送的目标功率变化过大，瞬态变化不要超过50W。

对数据进行变换的代码示例

有具体要发送的数据之后，这里操作的顺序和上面接收的顺序（接收->反变换）相反，先变换后转化为uint8_t进行CAN通信发送。

下面是进行变换的示例代码：

```
1  //存放数据的结构体定义（假设该结构体中有要发送的原始数据
2  typedef struct
3  {
4      float Set_Power;          //底盘当前功率
5      float data2;              //未定
6      float data3;              //未定
7      float data4;              //未定
8  }PowerCtrl_True;
9  PowerCtrl_True Cap_SendData_True;
10
11 typedef struct
12 {
13     uint16_t Set_Power; //底盘当前功率
14     uint16_t data2;      //未定
15     uint16_t data3;      //未定
16     uint16_t data4;      //未定
17 }PowerCtrl;
18 PowerCtrl Cap_SendData;
19
20 //变换函数
21 void Cap_Data_Encode(void)
22 {
23     Cap_SendData.Set_Power = Cap_SendData_True.Set_Power *10 + 0x7FFF;
24     //剩余三个数据还没有定义，后面有再补充
25 }
```

底盘主控发送代码示例（使用低位在前逻辑）

由底盘主控发送、超电主控接收的数据，采用和上面所讲述的一样的数据格式，也就是四个uint16_t元素组成的数组，然后以强制类型转换或者低位在前逻辑转化为CAN实际发送使用的uint8_t类型元素进行数据传输。这里仅展示后者。

这里沿用上面的Cap_Send_Data结构体，进行发送。

```
1  uint8_t Cap_CANSendData[8];
2
```

```
3 //类型转换
4 Cap_CANSendData[0] = Cap_SendData.Set_Power;
5 Cap_CANSendData[1] = Cap_SendData.Set_Power >> 8;
6 Cap_CANSendData[2] = 0; //其余三个未定数据这里省略处理
7 Cap_CANSendData[3] = 0;
8 Cap_CANSendData[4] = 0;
9 Cap_CANSendData[5] = 0;
10 Cap_CANSendData[6] = 0;
11 Cap_CANSendData[7] = 0;
12
13 //发送函数
14 HAL_CAN_AddTxMessage(&hcan, header, Cap_CANSendData, send_mail_box);
15 //一样，这里按照自己的进行配置
```

Version 0.5

添加章节编码，增加电路拓扑图;超电第三位发送数据改为电管功率

对应超电硬件V0.1，软件V2.7左右

写于25/1/13